

across service seekers (the citizens and businesses) and various service providers (government departments) and even among government departments.

To test the representative tools, we configured each tool to run the test. The configuration included installation, setting up test parameters, test data, result analysis, etc.

We ran the test on the machines. Configuration for the machine was Intel core i3-3110M 2.40GHz processor with 2GB RAM, running Microsoft Windows. The tests were conducted two times a day to minimise the effect of the Internet connection on test results and to obtain accurate measurements.

## Overview of tools

**Wapiti.** Wapiti is a web application vulnerability scanner that was created in 2006 by Nicolas Surribus. It does not scan the source code of the application but scans the web pages of the launched web application. Wapiti acts like a fuzzer and injects payloads to see if a script is vulnerable. Wapiti can detect lots of vulnerabilities, such as file-handling errors, database injection, cross-site scripting, LDAP injection and CRLF injection. Wapiti is easy to use, open source and the user does not need security knowledge. But it is not able to find all the vulnerabilities.

**OWASP ZAP.** OWASP ZAP is a penetration-testing tool which comes with plenty of features. Its main features are active scanning which is used to find certain kind of vulnerabilities, including XSS and others, except some logical vulnerabilities that can never be found by any automated security testing tool. Another interesting feature of ZAP is fuzzing. ZAP provides a list of fuzzers with the help of which we can fuzz any part of the application. This tool is open source, easy to use, well organised and up-to-date. High false positive factors and zero support for multiple scanning profiles are its main disadvantages.

**Netsparker Community Edition.** Netsparker is a free web application security scanner. It helps the developers

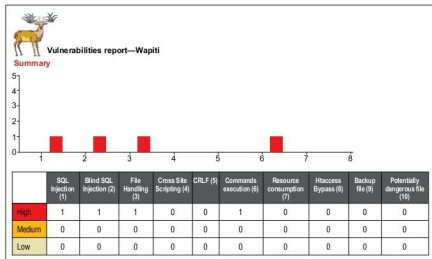


Fig. 3: Wapiti test results

who want to scan their web applications instantly and find the vulnerabilities. Netsparker Community Edition scans for SQL injection, XSS, Boolean SQL injection, backup files and static tests. It scans all types of web applications without limiting itself to technology or platform they are built on. It supports Javascripts/AJAX, can show impact of vulnerabilities or remedy for that vulnerability.

## Tools results and analysis

The web application was tested using the three tools OWASP ZAP, Wapiti and Netsparker Community Edition. The test results obtained using these tools were:

**Testing results with Wapiti.** Using Wapiti, the following four vulnerabilities were discovered in the Jammu & Kashmir application (the number in brackets refers to the number of occurrence of the vulnerability):

**SQL injection (1):** SQL injection is a technique that exploits a vulnerability occurring in the database of an application. It is a code injection that can allow the tester to read sensitive information from database and to modify it. SQL Injection occurs when input contains escaped characters, or user input is not strongly typed.

**Blind SQL injection (1):** Blind injection occurs when SQL injection is already present in the application but its result is not visible. Escaped characters

should be handled properly to protect an application from SQL injection.

**File handling (1):** The attack is known as path traversal or directory traversal. Its aim is to access files which are stored outside the web root folder. The attacker tries to explore the directories stored in web server.

**Command execution (1):** This attack consists of executing system commands on the server. The attacker tries to inject this command in the request parameter.

## Test results with OWASP ZAP

Using the active scanning feature of OWASP ZAP, 17 potential vulnerabilities were found, as mentioned below (the number following the vulnerability refers to the number of occurrence of the vulnerability):

**Cross site request forgery (2).** Also known as one-click attack or session riding, and abbreviated as CSRF or XSRF, it is a type of malicious exploit of a website whereby unauthorised commands are transmitted from a user that the website trusts.

**Cookie set without HttpOnly flag (1).** If a cookie has been set without the HttpOnly flag, it means the cookie can be accessed by JavaScript. If a malicious script can be run on this page then the cookie will be accessible and can be transmitted to another site. If this is a session cookie then session